Lecture 26.11: K-Nearest Neighbors (KNN) from Scratch — Metric Space & Lazy Learning Formulation

📌 Executive Summary

Instance-based non-parametric learning theory, Euclidean distance geometry, hyperparameter k selection, and the critical requirement of feature scaling.

🔬 Mathematical Formulation & Theory

1. The Lazy Learning / Instance-Based Paradigm

Unlike Linear and Logistic Regression which are **parametric eager learners** (they estimate a fixed vector of weights during training and discard the data), K-Nearest Neighbors is a **non-parametric lazy learner**:

- **Training Phase (`fit`)**: Requires $\mathcal{O}(1)$ time. The algorithm simply stores the training instances in memory.
- **Inference Phase (`predict`)**: Requires $\mathcal{O}(m \cdot d)$ compute per query point, as the distance between the query and all m training points must be computed and sorted!

2. Distance Metrics in Euclidean Space

In d-dimensional space, the Euclidean distance (L_2 norm) between query point $\mathbf{x}$ and training sample $\mathbf{x}^{(i)}$ is:

$$d(\mathbf{x}, \mathbf{x}^{(i)}) = \sqrt{\sum_{j=1}^{d} \left(x_j - x_j^{(i)} \right)^2} = \|\mathbf{x} - \mathbf{x}^{(i)}\|_2$$

3. Hyperparameter k Bias-Variance Trade-off

- $k = 1$: The model assigns the label of the single closest training instance.
  - Decision boundaries are complex, jagged Voronoi polygons.
  - Training accuracy is 100%, but the model suffers from **high variance (overfitting)** and is vulnerable to noise.

- **Large k (e.g. $k = m$)**: The model predicts the global majority class regardless of the query point's location (**high bias / underfitting**).
- **Rule of Thumb**: Set $k = \sqrt{m}$, and select an **odd number** for binary classification to eliminate tie-breaking deadlocks.

💻 Python & NumPy Implementation

```
import numpy as np

def euclidean_distance(x1, x2):
    return np.sqrt(np.sum((x1 - x2) ** 2))

# Verify distance computation
p1 = np.array([1.0, 2.0])
p2 = np.array([4.0, 6.0])
dist = euclidean_distance(p1, p2)
print(f"Distance between {p1} and {p2}: {dist:.4f} (Expected: 5.0)")
```

🎯 Key Takeaways & Best Practices

1. **First-Principles Transparency**: Building from scratch demystifies optimizer mechanics, memory layouts, and algorithmic failure modes.
2. **Vectorization vs Loops**: Always eliminate Python `for` loops across sample dimensions by relying on BLAS-accelerated NumPy array operations.
3. **Shape Integrity**: Maintain explicit assertions on input array ranks and shapes (`X.shape == (m, d)` , `y.shape == (m,)`) to prevent silent broadcasting bugs.